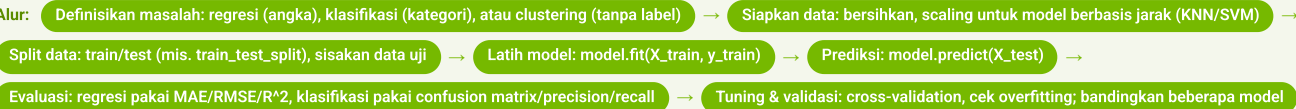


Machine Learning Fundamentals



3 Jenis ML

- Supervised: data berlabel (regresi + klasifikasi)
- Unsupervised: tanpa label (clustering)
- Reinforcement: hanya konteks, tidak dibahas
- Time series = supervised dengan input waktu

► Menentukan kategori masalah

Linear Regression

- Cari garis terbaik: $y = w \cdot x + b$
- w = kemiringan, b = titik awal (intercept)
- Output angka kontinu

```
from sklearn.linear_model import LinearRegression
m = LinearRegression().fit(X, y)
```

► Prediksi nilai numerik (harga, suhu)

Logistic Regression

- Output sigmoid: probabilitas 0..1
- Threshold (default 0.5) -> kelas Ya/Tidak
- Batas keputusan garis lurus, mudah diinterpretasi

```
from sklearn.linear_model import LogisticRegression
m = LogisticRegression().fit(X, y)
```

► Klasifikasi biner / probabilitas

Decision Tree & KNN

- Tree: rantai if/else, batas kotak-kotak, tak perlu scaling
- KNN: lihat K tetangga terdekat, mayoritas menang
- KNN perlu scaling, lambat saat predict

```
from sklearn.neighbors import KNeighborsClassifier
m = KNeighborsClassifier(n_neighbors=5)
```

► Tree: data kecil/interpretasi; KNN: batas berliku

SVM

- Cari margin terlebar antar kelas
- Support vectors = titik penentu batas
- kernel='linear' (lurus) atau 'rbf' (melengkung)
- Perlu scaling

```
from sklearn.svm import SVC
m = SVC(kernel='rbf')
```

► Batas kompleks, dataset menengah

Ensemble: Voting & Bagging

- Hard voting: 1 model = 1 suara, mayoritas menang
- Soft voting: rata-rata probabilitas (biasanya lebih akurat)
- Bagging = Bootstrap Aggregating
- Random Forest: banyak pohon, data + fitur acak; tahan noise

```
from sklearn.ensemble import RandomForestClassifier
m = RandomForestClassifier() # n_estimators=100 default
```

► Mau lebih stabil & akurat dari 1 model

Boosting & XGBoost

- Boosting: model berurutan, perbaiki kesalahan sebelumnya
- XGBoost = eXtreme Gradient Boosting (super cepat)
- Built-in regularisasi (anti overfitting), tangani data kosong
- Dominan di kompetisi Kaggle; NVIDIA dukung GPU-nya

```
from xgboost import XGBClassifier
m = XGBClassifier()
```

► Dataset bersih/tabular; kejar akurasi tinggi

Clustering

- K-Means: tentukan K, cluster bulat, tak deteksi outlier
- DBSCAN: berbasis kepadatan, tak perlu K, deteksi outlier
- Hierarchical: gabung cluster terdekat, K opsional
- Pilih K: Elbow Method (titik siku) & Silhouette Score

```
from sklearn.cluster import KMeans, DBSCAN
labels = KMeans(n_clusters=3).fit_predict(X)
```

► Data tanpa label (segmentasi, grouping)

Time Series & SARIMA

- Data berurutan waktu: ada trend + seasonality + noise
- Urutan waktu penting (beda dari regresi biasa)
- SARIMA = Seasonal AutoRegressive Integrated Moving Average
- Menangkap pola musiman otomatis untuk forecast

► Prediksi masa depan dari data historis

Overfitting & Cross-Validation

- Underfitting: model terlalu sederhana (jelek di train & test)
- Overfitting: terlalu rumit (bagus di train, jelek di test)
- Good fit: pas, generalisasi baik
- Cross-validation: bagi data jadi beberapa fold untuk uji adil

```
from sklearn.model_selection import cross_val_score
scores = cross_val_score(m, X, y, cv=5)
```

► Memastikan model tak hafal data latih

GPU: cuML & XGBoost

- RAPIDS cuML: API identik scikit-learn, 10-50x lebih cepat
- Cukup ganti import: sklearn.* -> cuml.*
- XGBoost: device='cpu' -> device='cuda'
- cuDF = Pandas di GPU; ONNX + Triton untuk deployment

```
from cuml.ensemble import RandomForestClassifier
XGBClassifier(device='cuda')
```

► Dataset besar, butuh training cepat

Quick-Ref Model Klasifikasi

Model	Batas Keputusan	Scaling?	Kecepatan	Interpretasi
Logistic Reg.	Garis lurus	Ya	Cepat	Mudah
Decision Tree	Kotak-kotak	Tidak	Cepat	Mudah
KNN	Berliku	Ya	Lambat	Sulit
SVM Linear	Garis lurus	Ya	Sedang	Sedang
SVM RBF	Melengkung	Ya	Lambat	Sulit

Istilah Kunci

MAE / RMSE / R^2 — Metrik regresi: MAE & RMSE -> 0 (RMSE penalti besar untuk error besar); R^2 -> 1.0 = makin baik. MAPE = error persen (ideal < 10%).

Confusion Matrix — Tabel TP/TN/FP/FN. FP = alarm palsu, FN = kasus terlewat (sering paling berbahaya).

Precision / Recall — Precision = dari yang diprediksi positif, berapa benar. Recall = dari yang benar-benar positif, berapa tertangkap.

Threshold — Batas probabilitas (default 0.5) untuk mengubah output sigmoid jadi kelas Ya/Tidak.

Bagging vs Boosting — Bagging: pohon paralel dari data acak (Random Forest, tahan noise). Boosting: model berurutan saling memperbaiki (XGBoost, butuh data bersih).

Support Vectors — Titik data terdekat ke batas keputusan SVM yang menentukan posisi margin terlebar.

Elbow & Silhouette — Cara pilih jumlah cluster K: Elbow cari titik siku kurva inertia; Silhouette pilih skor tertinggi.

No Free Lunch — Tidak ada satu model terbaik untuk semua masalah; selalu coba & bandingkan beberapa model.